

SDS 2412 — Large-Scale Book Recommendation System

Complete Notebook Walkthrough

SETUP CELL — Environment & Spark Initialization

This is the first cell that runs before anything else. It installs PySpark, which is the Python interface to Apache Spark — the distributed computing engine that powers the whole system. Think of Spark as the engine of a truck; pandas alone is like a bicycle, fine for small loads but not built for scale.

After installing, we import our core libraries: pandas for data manipulation, numpy for numerical operations, and PySpark's SQL and functions modules for distributed processing. We then start a Spark Session — this is like turning the engine on. The configuration sets the driver memory to 4GB and shuffle partitions to 50, meaning Spark will use up to 4GB of RAM and split shuffle operations into 50 parallel chunks. In a real cluster these numbers would be much larger, but on a single machine this keeps things manageable. The final print confirms Spark started correctly and shows its version number.

MILESTONE 1 — DATA FOUNDATIONS & SYSTEM ARCHITECTURE

Cell 1.1 — Loading the Raw Data

This is where the data enters the system for the first time. We load three separate CSV files from the Book-Crossing dataset: Books.csv, Ratings.csv, and Users.csv. Each file represents a different entity in our system.

Two important parameters here. First, `encoding='latin-1'` — the dataset contains book titles and author names from many countries, with accented characters like é, ü, ñ. The default UTF-8 encoding cannot read these, so we switch to latin-1 which covers the full Western character set. Second, `on_bad_lines='skip'` — some rows in the original CSV are malformed (extra commas, line breaks inside fields). Instead of crashing the entire load, we simply skip those broken rows.

The print output shows us the shape of each table. This is the first checkpoint — confirming the data arrived intact before we do anything to it.

Cell 1.2 — Previewing the Data Structure

Before cleaning anything, we look at what we are actually working with. This cell prints the first three rows of each table and lists all column names with their data types.

This matters because real datasets almost never look exactly as described. By previewing first, we can see things like unexpected column names, values that are in the wrong column, or data types that look wrong. It is the equivalent of reading a document before editing it. The output here feeds directly into our decision-making for the cleaning steps that follow.

Cell 1.3 — Data Characterisation: The 3 Vs

This cell is the formal analysis required for Milestone 1 — characterising the dataset across Volume, Velocity, and Variety.

Volume is straightforward: we count total ratings (over 1.1 million), unique users, and unique books. But the more revealing number is the sparsity calculation. The total possible interactions would be every user rating every book. The actual interactions are only the ratings we have. The ratio between these two numbers tells us the matrix is about 99.8% empty — meaning the vast majority of user-book combinations have no rating at all. This sparsity is not a problem with our dataset; it is the fundamental challenge of all recommendation systems. If every user had rated every book, recommendations would be trivial.

Variety looks at what types of data we have: structured numerical ratings, text fields like author and publisher, categorical data like country, and metadata like publication year.

Velocity in the context of this dataset refers to how fast new ratings would arrive in a live production system — addressed properly when we build the streaming layer in Milestone 3.

Cell 1.4 — Fixing the Year Column

This cell addresses a specific data quality issue found during preview. The Year-Of-Publication column in the raw Books.csv contains some rows where a publisher name or other text accidentally ended up in the year field. This happens in real datasets when the original data entry had formatting errors.

`pd.to_numeric(..., errors='coerce')` attempts to convert every value in the column to a number. Anything that cannot be converted — like a publisher name — becomes NaN (Not a Number) instead of crashing the program. We then filter to only keep years between 1800 and 2024, removing clearly invalid values like year 0 or year 9999 that also appear in the raw data.

Cell 1.5 — Missing Values & Implicit Ratings

This cell does two things. First it prints a count of missing values in every column across all three tables — this is our data quality report before cleaning begins.

Second, and more importantly for the recommendation system, it identifies and quantifies implicit vs explicit ratings. This distinction is fundamental to how the system works:

Explicit ratings are scores from 1 to 10 that a user deliberately gave a book. They directly express an opinion. A rating of 9 means the user liked the book. A rating of 2 means they did not.

Implicit ratings are ratings of 0 in this dataset. They do not mean the user disliked the book. They mean the user interacted with the book — perhaps browsed it, added it to a list, or the system recorded the interaction — but gave no score. They are a weaker signal.

Approximately 70% of all rows have a rating of 0. We separate these into two groups because they require different treatment. The explicit ratings train our recommendation model. The implicit ratings carry information but cannot be used the same way, because a 0 does not tell us whether someone loved or hated a book.

Cell 1.6 — Cleaning the Books Table

Three cleaning operations happen here in sequence.

First, we drop any book row missing an author or publisher. Without these fields a book cannot participate in content-based filtering later, so keeping it would create incomplete records.

Second, we remove the three image URL columns (Image-URL-S, Image-URL-M, Image-URL-L). These were links to book cover images on Amazon. They are not useful for any of our ML models and would just waste memory and storage.

Third, we re-apply the year conversion from Cell 1.4 on the clean subset and convert the result to integers, filling any remaining nulls with 0. This gives us a clean, complete books table ready for analysis and modelling.

Cell 1.7 — Cleaning Ratings and Users

This cell handles the ratings and users tables.

For ratings, we use the explicit/implicit distinction identified in Cell 1.5 to create two separate dataframes. `explicit_ratings` contains only rows where the rating is 1–10.

`implicit_ratings` keeps the 0s. All ML training in Milestone 4 will use `explicit_ratings` only.

For users, we face a decision about missing ages. About 40% of users have no age recorded. If we simply dropped every row with a missing age, we would lose nearly half our user base — which would severely hurt the recommendation model. Instead, we fill missing ages with the median age of users who did provide it. The median is preferred over the mean here because age data in this dataset has outliers (some entries have ages like 200 or 0), and the median is not affected by extreme values the way the mean is.

We also extract the country from the Location field. The raw location is a freeform string like "chicago, illinois, usa" — we take everything after the last comma as the country. This gives us a usable geographic feature.

Cell 1.8 — Building the Working Dataset

This cell brings everything together for the first time. We merge the three cleaned tables into one flat working dataframe using two join operations.

First, `explicit_ratings` joins to `books_clean` on the ISBN column — attaching all book metadata to each rating event. Second, that result joins to `users_clean` on the User-ID column — attaching user age and country to each rating. The join type is inner, meaning we only keep rows where a match exists in all three tables. Any rating for a book we do not have in our books table, or from a user we do not have in our users table, gets dropped.

The result is our master working dataset: one row per rating event, with everything we know about that book and that user attached to it. This single table is what flows into the Parquet save step, the Spark loading step, and ultimately into the ML pipeline in Milestone 4.

Cell 1.9 — Saving to Parquet

This cell is the actual data ingestion pipeline deliverable for Milestone 1. We save all five cleaned and processed dataframes to Parquet format.

What is Parquet? Parquet is a columnar storage file format designed for large-scale data systems. Unlike a CSV which stores data row by row, Parquet stores data column by column. This matters enormously for analytics. If you want to compute the average book rating across all users, a CSV has to read every row and pick out the rating field. Parquet reads only the rating column and skips everything else. This makes queries dramatically faster and the files significantly smaller than equivalent CSVs — typically 5 to 10 times smaller due to compression. Parquet is the standard file format in real distributed systems built on HDFS (Hadoop), S3 (Amazon), and Azure Data Lake. By saving our data in Parquet here, we are already following production-grade data engineering practice.

We also create separate files for books, users, explicit ratings, implicit ratings, and the full working dataset. Each file serves a different downstream purpose — the explicit ratings go to Spark processing in M2, the working dataset goes to ML in M4.

Cell 1.10 — Loading into Spark

This cell is the boundary between single-machine processing and distributed processing. Everything before this ran in pandas on one machine. From here, the data lives inside Spark.

We load each Parquet file as a Spark DataFrame. In a real multi-node cluster, Spark would automatically distribute these partitions across physical machines — each machine would hold and process a slice of the data in parallel. On our single machine it simulates this behaviour. The partition count printout shows how many parallel chunks each dataset is split into. `printSchema()` confirms that Spark correctly inferred all column types from the Parquet metadata — one of the advantages of Parquet over CSV, which has no type information embedded.

Cell 1.11 — First Spark Query

This cell serves as proof that the full pipeline works end to end — from raw CSV through cleaning through Parquet through Spark loading — and that Spark can actually execute queries against our data.

We register the working dataset as a SQL view called "interactions" and run a GROUP BY query to find the top 10 most rated books. This is also our first meaningful result: it tells us which books in the dataset have the highest engagement, which is a basic but important recommendation signal. The same query written here would work identically on 500 million rows — the only difference would be that Spark would distribute the computation across more machines. That scalability without code change is the core value of the Spark architecture.

MILESTONE 2 — DISTRIBUTED DATA PROCESSING

Cell 2.1 — Repartitioning (Sharding)

This cell demonstrates data partitioning, one of the core concepts of distributed systems. We repartition the ratings data into 10 partitions by User-ID.

Partitioning means dividing the full dataset into separate chunks so different machines can work on different chunks simultaneously. By partitioning on User-ID, all ratings from the same user go

to the same partition — this is called hash partitioning. In a real cluster with 10 machines, partition 0 lives on machine 0, partition 1 on machine 1, and so on. When a query involves a specific user, only the machine holding that user's partition needs to respond.

The `mapPartitionsWithIndex` section then inspects how many rows ended up in each of the 10 partitions. We are looking for even distribution. If one partition has 80% of the data and the others share 20%, that one machine becomes a bottleneck — this is called data skew. Even distribution means the load is balanced across all nodes.

Cell 2.2 — Batch Aggregations

This is the core of Milestone 2: running large-scale batch processing jobs using Spark SQL. We run two separate jobs.

Job 1 computes a user profile for every user: how many books they have rated, their average rating, and their minimum and maximum ratings. This profile captures each user's behaviour and taste calibration — some users rate everything a 9 while others are harsh and average a 4. Knowing this helps the recommendation model.

Job 2 computes an item profile for every book: how many times it has been rated and its average score. This becomes the popularity signal — a book with 5,000 ratings and an average of 8.5 is a very strong recommendation candidate.

Both of these jobs follow the MapReduce pattern. The Map step applies a function to each individual row independently (read each rating). The Reduce step aggregates those individual results (group by user, compute stats). Spark handles the parallelism automatically — across 10 partitions, 10 machines would run these jobs simultaneously.

These user and item profiles are not just output — they feed directly into the ML layer in Milestone 4 as pre-computed features.

Cell 2.3 — Save Outputs & Scalability Test

The first part saves the user profiles and item profiles to Parquet. These files are the link between the distributed processing layer and the machine learning layer — when we build the ALS model in Milestone 4, it will read these profiles as pre-computed inputs rather than recomputing them from raw data every time. This is exactly how production ML pipelines work: batch jobs run on a schedule, produce outputs, and the ML models consume those outputs.

The second part is our scalability analysis. We sample the ratings at 5 different sizes (10%, 25%, 50%, 75%, 100%) and measure how long the aggregation job takes at each size. The results let us fit a complexity curve and project: if the job takes T seconds on 430,000 rows, how

long would it take on 25 million rows? And how much would 10 Spark nodes reduce that time? The expected result is roughly linear scaling — double the data, roughly double the time — which is the best-case Big O behaviour for this type of aggregation job.

Cell 2.4 — Workflow Documentation

This cell documents the complete Milestone 2 pipeline in a structured format — every stage in sequence with a description of what it does. This is both a deliverable and a useful reference for understanding how the batch processing system hangs together.

The seven stages form a complete chain: raw CSV ingestion → cleaning → Parquet storage → Spark loading → hash partitioning by User-ID → batch MapReduce aggregation → profile storage. Each stage has a specific purpose and produces output that the next stage consumes. This is the definition of a pipeline — no stage can run before the one before it completes.

The fault tolerance section explains a critical Spark concept: RDD lineage. When Spark processes data, it does not constantly write intermediate results to disk. Instead, it remembers the sequence of transformations applied to the data (the lineage). If a partition fails — for example, because a machine crashes mid-job — Spark does not restart the entire job. It recomputes only the lost partition by replaying its lineage from the last stable point. In a production HDFS environment, every data block would additionally be replicated across 3 machines, meaning the data itself survives hardware failures. Our Parquet files on local disk serve the equivalent role in our single-machine setup.

MILESTONE 3 — STREAMING & REAL-TIME SYSTEMS

Cell 3.1 — Preparing the Event Stream

This cell sets up the simulation of a real-time data stream. In a production recommendation system, user ratings do not all arrive at once — they come in continuously, one event at a time, as users interact with the platform. The tool designed for this is Apache Kafka, a distributed message streaming platform. Kafka acts as a buffer between data producers (users rating books) and data consumers (our processing system). Producers push events into Kafka topics; consumers read from those topics and process each event.

We were not able to run a full Kafka broker in this environment, so we simulate the same behaviour in Python. We shuffle the working dataset randomly to remove any ordering, then replay it row by row as if each row were a Kafka message arriving in real time. The behaviour of our processing code is identical to what a Kafka consumer would do — the only difference is the source of the events.

Cell 3.2 — Stream Processor (Speed Layer)

This is the speed layer of our Lambda Architecture. It processes each event as it arrives and maintains four in-memory data structures that update continuously.

`recent_ratings` is a deque with a maximum size of 500 — this is our sliding window. A deque automatically drops the oldest item when it is full, so at any moment it holds exactly the last 500 events. This is how streaming systems handle recency: you do not care about what happened three months ago, only what is happening now.

`item_live_counts` tracks how many times each book has been rated in the current stream window — this drives the trending books output. `item_live_scores` accumulates the scores for each book so we can compute a live average. `user_live_history` tracks which books each user has interacted with during the stream — this feeds into the Bloom filter in the next cell.

We process 1,000 events and measure the latency of each one (how long it takes to process a single event) and the total throughput (events per second). These are the key metrics for evaluating a streaming system — latency tells you how responsive the system is, throughput tells you how much load it can handle.

Cell 3.3 — Bloom Filter

This cell implements one of the approximate algorithms required by Milestone 3.

The problem it solves: before recommending a book to a user, we need to check whether they have already read it. The naive approach is to search through the user's full reading history every time. At scale — millions of users, thousands of books each — this becomes prohibitively slow.

A Bloom filter solves this with a completely different approach. Internally it is just an array of bits, all starting at 0. When we add an item, we run it through multiple hash functions, each producing a position in the bit array, and set those bits to 1. When we check whether an item has been seen, we run it through the same hash functions and check those same positions. If any bit is 0, the item has definitely never been added — no false negatives are possible. If all bits are 1, the item has probably been added — but there is a small chance that other items happened to set those same bits, producing a false positive.

Why accept false positives? Because the trade-off is enormous: constant time $O(1)$ lookups and a fraction of the memory that a dictionary or set would require. In our case, a filter for 500 books takes only 625 bytes. A dictionary of 500 string ISBNs would take around 25 kilobytes. At

millions of users, that difference is the gap between a system that fits in memory and one that does not.

We build one Bloom filter per user from their streaming history, then test it against a known book (should return true) and a fake ISBN (should return false).

Cell 3.4 — Batch vs Streaming Comparison

This comparison table is the analytical conclusion of Milestone 3. It directly answers why we need both layers in our Lambda Architecture rather than choosing one or the other.

Batch (Spark, Milestone 2): Processes the complete 433,000 rating dataset. Produces exact results because it sees all the data. But it takes time to run — in production you would schedule it nightly — meaning recommendations can be hours out of date. Memory is not a concern because data is stored on disk in Parquet files.

Streaming (our simulator, Milestone 3): Processes only the last 500 events in the sliding window. Results are approximate because you only see a small slice of history. But latency per event is under 0.1 milliseconds and throughput is around 15,000 events per second — meaning the system reacts to user behaviour almost instantly. Memory is the constraint because all state lives in RAM.

The Lambda Architecture conclusion is that neither layer alone is sufficient. Batch is accurate but stale. Streaming is fresh but approximate. The serving layer — which we build in Milestone 5 — merges both: use the batch model for the bulk of recommendations, use the streaming layer to filter out books just rated and boost currently trending titles. Together they give you both accuracy and recency.

MILESTONE 4 — SCALABLE MACHINE LEARNING

Cell 4.1 — Feature Engineering for ALS

Before we can train the recommendation model, we need to prepare the data in the format the algorithm requires.

ALS (Alternating Least Squares) requires all IDs to be non-negative integers. User-ID in our dataset is already numeric. But ISBN is a string — a 10 or 13 character alphanumeric code. Spark's StringIndexer converts each unique ISBN string to a unique integer, starting from 0 for the most frequent. This mapping is stored in the isbn_model so we can reverse it later — turning predicted book indices back into actual ISBNs for display.

We also cast all relevant columns to the correct numeric types: `user_index` as Integer, `book_index` as Integer, and the rating as Float. ALS requires Float for the rating column specifically.

Finally we split 80% of the data for training and 20% for testing. The `seed=42` ensures the split is reproducible — running the cell again produces the exact same train/test division, which matters for fair comparison of different model configurations.

Cell 4.2 — Collaborative Filtering with ALS

What is ALS? ALS stands for Alternating Least Squares. It is our Collaborative Filtering (CF) algorithm — meaning it makes recommendations based purely on the pattern of who rated what, without needing to know anything about what the books are actually about. When you hear CF mentioned in this project, this is the cell it refers to. CF and ALS are the same thing here — CF is the approach, ALS is the algorithm we used to implement it.

The core idea is matrix factorisation. Imagine a giant matrix where rows are users and columns are books, and each cell is the rating that user gave that book. The matrix is 99.8% empty. ALS factorises this sparse matrix into two smaller dense matrices: a Users matrix (users × latent factors) and an Items matrix (books × latent factors). Latent factors are hidden dimensions the model discovers automatically — factor 7 might loosely correspond to "literary fiction" or "fast pacing" without ever being told that. The model learns to use these factors to predict what rating a user would give a book they have never seen.

The "Alternating" part describes the training process: fix the Items matrix and solve for the Users matrix using least squares, then fix the Users matrix and solve for the Items matrix. Alternate until predictions stop improving. `maxIter=10` means 10 of these alternations. `rank=50` means 50 latent factors. `regParam=0.1` prevents overfitting — without it the model would memorise training data rather than learning general patterns. `coldStartStrategy="drop"` handles users or books that appear in the test set but were not seen during training — rather than making a nonsensical prediction, those rows are simply dropped from evaluation.

RMSE (Root Mean Squared Error) and MAE (Mean Absolute Error) both measure prediction error on the 1–10 rating scale. RMSE penalises large errors more heavily because it squares them before averaging. MAE is the straightforward average error — a MAE of 1.4 means predictions are off by 1.4 rating points on average. Both are reported here so we have a complete picture of how the model performs.

Cell 4.3 — Content-Based Filtering

ALS is powerful but blind to content. It cannot distinguish between two books that happen to have similar rating patterns but are completely different genres. Content-based filtering fills this gap by using what we actually know about the books themselves — their author, publisher, and publication year.

We build a content string for each book by concatenating these three fields into a single text field. We then apply TF-IDF (Term Frequency-Inverse Document Frequency) vectorisation. TF-IDF assigns a score to each word relative to how distinctive it is across the whole dataset: a word that appears frequently in one book but rarely across all books gets a high score and becomes a strong identifying feature. A word that appears in almost every book gets scored near zero because it tells us nothing distinctive.

The result is a matrix of shape (number of books $\times$ 5,000 features). To find books similar to one a user liked, we compute cosine similarity between that book's feature vector and every other book's vector. Cosine similarity measures the angle between two vectors — vectors pointing in a similar direction score close to 1, completely unrelated books score near 0.

The `get_cb_recommendations` function wraps this: given a book title, find its TF-IDF vector, compute similarity to all other books, and return the top N most similar. This content-based component feeds directly into the hybrid cell that follows.

Cell 4.4 — Hybrid Recommender

This is where the two models are actually combined into a working recommendation engine. The previous two cells built the ingredients separately — ALS produced collaborative filtering scores based on user behaviour, TF-IDF produced content similarity scores based on book metadata. This cell mixes them together and produces real, human-readable book recommendations.

The `hybrid_recommend` function takes a user ID and a seed book title and works in four steps. Step 1: ask the trained ALS model for its top 50 book recommendations for that specific user — these are the collaborative filtering scores reflecting what users with similar taste have liked. Step 2: take the seed book and compute content similarity scores against all other books using the TF-IDF matrix from Cell 4.3. Step 3: normalise both sets of scores to the same 0–1 scale so they can be fairly compared, then blend them — 70% weight on the ALS score and 30% on the content similarity score. ALS gets higher weight because with over 430,000 explicit ratings it carries a much stronger signal. Content-based fills in where ALS is weak, particularly for books with fewer ratings where the collaborative signal is thin. Step 4: map the book indices back to actual ISBNs and then to real book titles and authors so the output is human-readable.

We tested this on a real user from the dataset — user 276788 — who has rated Harry Potter, using Harry Potter as the seed book. The output shows 10 recommended books with three

score columns: the ALS contribution, the content-based contribution, and the final blended hybrid score.

Looking at the actual output, the ALS signal dominates across all 10 results. The content-based score (`cb_norm`) is near zero for most recommendations, with only one book — Mount Misery — receiving a non-zero content score of 0.074. This is expected and explainable: the hybrid function only computes content similarity for books that ALS has already shortlisted, and most of those ALS recommendations — a Portuguese psychology book, Neil Gaiman's Sandman, Art Spiegelman's Maus — share almost no metadata with Harry Potter. Their TF-IDF vectors point in completely different directions, producing near-zero cosine similarity scores. This does not mean the hybrid is broken — it means ALS has strong enough data for this particular active user that the content signal is not needed to steer the results. In a production system, the content-based weight would be increased for new users with few ratings, where ALS has little data to work with, and reduced for active users like this one where collaborative filtering is already performing well.

You will also notice some titles display with encoding errors — characters like `Ã©` and `Ã¼` appearing in foreign language titles. This is a display artefact from the latin-1 encoding of the original dataset and does not affect the model or the scores in any way.

Cell 4.5 — Evaluation: Precision@10

This cell evaluates the overall quality of the recommendation model across 200 users and prints the complete M4 summary.

Rather than running a separate Spark query for each test user — which would mean 200 round trips to the Spark engine — we pull all test predictions into pandas in a single job and evaluate entirely in pandas. This is a deliberate optimisation that makes the evaluation significantly faster.

`batch_precision_at_k` calculates Precision@10 for each user. For a given user: take the top 10 books the model predicted they would like most. Count how many of those 10 are books the user actually rated 7 or above in the test set. That ratio — hits divided by 10 — is that user's Precision@10 score. Averaging across all evaluated users gives the overall metric. A Precision@10 of 0.15 means that on average, 1.5 out of every 10 recommendations are books the user genuinely liked.

The final summary block prints the complete picture of all four components: ALS collaborative filtering with its RMSE and MAE scores, content-based TF-IDF with its 5,000 features, the hybrid weighting, and the Precision@10 ranking quality score. This is the full M4 deliverable in one place — three models built, combined, demonstrated on real data, and evaluated with a meaningful metric.

MILESTONE 5 — SYSTEM OPTIMIZATION & DEPLOYMENT

Cell 5.1 — LRU Cache (Recommendation Caching)

This cell solves a practical performance problem. The hybrid recommendation function is expensive — it calls the ALS model, computes cosine similarity, and runs several Spark operations. If the same user requests recommendations twice, it makes no sense to recompute everything from scratch.

The solution is an LRU (Least Recently Used) cache. It stores the results of recent recommendation requests in memory. When a request comes in, the cache checks if it already has the answer — if yes, it returns it instantly without touching the model. If no, it computes the result, stores it, and returns it.

The "Least Recently Used" part refers to what happens when the cache is full. The capacity is set to 100 entries. When a 101st entry needs to be stored, the cache automatically evicts the entry that was accessed least recently — the assumption being that recently used results are more likely to be needed again than old ones.

We simulate 50 requests across 20 users — a realistic mix of repeat requests and new ones. The output shows the average latency with and without cache. The cached path returns in a fraction of a millisecond because it is just a dictionary lookup. The uncached path goes through the full model pipeline. In a production system this cache would be Redis — a dedicated in-memory database designed exactly for this purpose.

Cell 5.2 — Model Monitor & Drift Detection

This cell implements monitoring — the system watching itself to detect when the model starts performing worse.

The `ModelMonitor` class maintains a rolling window of the last 100 prediction errors. Every 50 predictions it computes the rolling MAE across that window and compares it to a drift threshold of 3.0. If the rolling MAE exceeds 3.0 — meaning predictions have become significantly less accurate — it fires a drift alert. In a production system a drift alert would trigger an automatic retraining job.

Why does drift happen? Because user behaviour and book popularity change over time. A model trained on last year's ratings may not reflect what users want today. The monitor catches this before it becomes a user-facing problem.

We feed 300 test predictions through the monitor and check whether the model is stable. The expected output is that rolling MAE stays around 1.99 and no drift alerts fire — meaning the model is performing consistently across the test set.

Cell 5.3 — Flask REST API

This cell deploys the entire recommendation system as a real HTTP API. Flask is a lightweight Python web framework. We define two endpoints.

`/recommend` takes three parameters in the URL: a `user_id`, an optional `seed_book`, and `n` (how many recommendations to return). It calls `get_recommendations` which goes through the cache first, then the hybrid model if needed. It returns a JSON response containing the recommendations, which layer served the request (cache or model), and the latency.

`/health` is a monitoring endpoint. Any production system exposes a health check — it tells infrastructure tools whether the service is running correctly. Ours reports the model name, current cache hit rate, and rolling MAE.

The API runs in a background thread so the notebook stays interactive. We then make actual HTTP requests to it using Python's `urllib` library — this confirms the API is genuinely running and serving real responses, not just simulated. This is the closest thing in the notebook to a real deployed system.

Cell 5.4 — Optimization Report

This cell prints the complete M5 optimization summary across four areas.

Caching shows the LRU hit rate and speedup factor from serving cached results versus computing fresh ones. Partitioning confirms the 10 Spark shards from M2 remain balanced with no data skew. Monitoring confirms the rolling MAE is stable at approximately 1.99 across 300 predictions with no drift detected. The NoSQL note is forward-looking — it describes how in a production M6 system, the cache would move to Redis and user profiles would move to MongoDB, both integrating with the serving layer.

MILESTONE 6 — INTEGRATED INTELLIGENT SYSTEM & CAPSTONE

Cell 6.1 — End-to-End Pipeline Function

This is the capstone cell — the one function that activates every layer we have built across all five milestones simultaneously, in sequence, for a single user request.

`end_to_end_pipeline` works in four layers. Layer 1 hits the batch layer — it looks up the user's profile in the working dataset to confirm they exist and count how many books they have rated. This is the M1/M2 output. Layer 2 hits the streaming layer — it retrieves the user's Bloom filter and live history from M3. Layer 3 hits the ML layer — it calls `get_recommendations` which goes through the M5 cache first and falls back to the M4 hybrid model if needed. Layer 4 applies the system-level innovation.

The innovation is MMR — Maximal Marginal Relevance. This addresses a known problem with pure collaborative filtering called the filter bubble. When ALS recommends books, it tends to recommend the most popular books from the most popular authors repeatedly. A user who liked one J.K. Rowling book might get five J.K. Rowling recommendations in their top 10. MMR fixes this by iteratively building the recommendation list: the first pick is the highest scoring book, but each subsequent pick is penalised if its author already appears in the selected list. The penalty of 0.3 means a book by a repeated author needs to score 0.3 higher than a diverse alternative to beat it. The result is a top 10 that is both relevant and varied across different authors.

The function logs the time spent in each layer so you can see exactly where the milliseconds go.

Cell 6.2 — System Evaluation

This cell runs the full end-to-end pipeline on 20 real users and measures performance across every layer. It prints a structured evaluation report showing the cache hit rate, average latency broken down by layer (batch lookup, stream lookup, ML inference, diversity injection), and the total average latency. It then repeats the M4 model metrics and the scale projection.

The scale projection is important for the presentation. The current system handles 433K ratings. Amazon's review dataset has 571 million — roughly 1,300 times larger. The projection shows that with 50 Spark nodes, batch jobs would complete in about 10 seconds and the stream layer would handle 750,000 events per second. The code itself does not change — only the infrastructure it runs on.

Cell 6.3 — System Summary

The final cell prints a complete summary of the entire project across all six milestones in one structured block. Every key number from every milestone is here — dataset size, sparsity, throughput, latency, RMSE, MAE, Precision@10, cache hit rate, speedup factor. This is your project's final output — the single printout that proves the full system ran end to end.

